

Ansible with the cp_mgmt Collection

Objectives

1. Install the check_point.mgmt collection and configure an Ansible inventory targeting the SMS.
2. Run a playbook twice and observe the idempotency behavior that distinguishes Ansible from Postman.
3. Install the check_point.mgmt collection and configure an Ansible inventory targeting the SMS.
4. Build a policy package end-to-end (package → sections → rules → publish → install) using only cp_mgmt modules.
5. Choose between mgmt_cli, Postman, and Ansible for a given customer scenario.

Lab Environment

Resource	Address	Credentials
SMS	10.1.1.100	admin / Cpwins!1
GW	10.1.1.111	admin / Cpwins!1
Jump server (Windows)	via MobaXterm bookmark	—

The lab is **pre-configured** by the instructor. The Management API listens on all IPs; the Standard policy package exists with GW as the target.

Pre-Flight

Ansible's check_point.mgmt collection wraps the same Management API you drove by hand in Labs 1–3 and through Postman in Lab 3. Two things make Ansible different from those tools: **idempotency** (running the same playbook twice does not double-apply) and **declarative state** (you describe what should exist, not how to get there). The session lifecycle from Lab 1 still happens — the collection's httpapi connection handles login and logout invisibly behind every task.

1. In MobaXterm, open the **Al-ubuntu** SSH session — address 10.1.3.33, login ubuntu / Cpwins!1. This Ubuntu host is your Ansible controller for this lab and Lab 6; the toolchain (ansible-core, check_point.mgmt, check_point.gaia, terraform, jq) is pre-installed.
2. Verify the pre-installed toolchain. Expect four version lines back — ansible-core 2.21+, terraform 1.15+, both check_point collections, jq 1.x:

```
ansible --version | head -1; terraform version | head -1; ansible-galaxy
collection list 2>/dev/null | grep check_point; jq --version
```

```
*** System restart required ***
Last login: Tue Jun  9 21:44:20 2026 from 10.1.3.200
ubuntu@AI-Ubuntu:~$ ansible --version | head -1; terraform version | head -1; ansible-galaxy collection list 2>/dev/null | grep check_point; jq --version
ansible [core 2.21.0]
  Terraform v1.15.5
    check_point.gala           7.0.0
    check_point.mgmt           6.9.0
    check_point.mgmt           6.9.0
    jq-1.8.2
```

3. Create the lab workspace:

```
mkdir -p ~/lab5 && cd ~/lab5
```

4. Write the inventory file. Paste the whole block:

```
cat > inventory.ini <<'EOF'
[checkpoint_mgmt]
sms ansible_host=10.1.1.100 ansible_user=admin ansible_password=Cpwins!1
[checkpoint_mgmt:vars]
ansible_network_os=checkpoint
ansible_connection=httpapi
ansible_httpapi_use_ssl=True
ansible_httpapi_validate_certs=False
EOF
```

5. Confirm using

```
ubuntu@AI-Ubuntu:~/lab5$ cat inventory.ini
[checkpoint_mgmt]
sms ansible_host=10.1.1.100 ansible_user=admin ansible_password=Cpwins!1
[checkpoint_mgmt:vars]
ansible_network_os=checkpoint
ansible_connection=httpapi
ansible_httpapi_use_ssl=True
ansible_httpapi_validate_certs=False
```

Task 1: First Playbook + Idempotency (15 Minutes)

1a — Write the playbook (paste this whole block):

```
cat > 01-add-host.yml <<'EOF'
- name: Lab 5 Task 1 - add a single host
  connection: httpapi
  hosts: checkpoint_mgmt
  gather_facts: false
  collections:
    - check_point.mgmt
  tasks:
    - name: Add lab-host-01
      cp_mgmt_host:
        name: lab-host-01
```

```
    ipv4_address: 10.20.30.50
    color: sea green
    comments: "Created by Lab 5 Task 1"
    state: present
- name: Publish
  cp_mgmt_publish:
EOF
```

```
ubuntu@AI-Ubuntu:~/lab5$ cat > 01-add-host.yml <<'EOF'
- name: Lab 5 Task 1 - add a single host
  connection: httpapi
  hosts: checkpoint_mgmt
  gather_facts: false
  collections:
    - check_point.mgmt
  tasks:
    - name: Add lab-host-01
      cp_mgmt_host:
        name: lab-host-01
        ipv4_address: 10.20.30.50
        color: sea green
        comments: "Created by Lab 5 Task 1"
        state: present
    - name: Publish
      cp_mgmt_publish:
EOF
```

1b — Run it:

```
ansible-playbook -i inventory.ini 01-add-host.yml
```

```
ubuntu@AI-Ubuntu:~/lab5$ ansible-playbook -i inventory.ini 01-add-host.yml
PLAY [Lab 5 Task 1 - add a single host] *****
TASK [Add lab-host-01] *****
changed: [sms]
TASK [Publish] *****
changed: [sms]
PLAY RECAP *****
sms                : ok=2    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Expect PLAY RECAP showing changed=2. In SmartConsole, lab-host-01 now appears under Network Objects.

1c — Run the SAME command again:

```
ansible-playbook -i inventory.ini 01-add-host.yml
```

Watch the recap. Second run reports ok=2 changed=1. The host already matches the desired state, so Ansible does nothing. Compare to Postman: re-sending add-host would error with "object already exists." That's **idempotency** — what makes Ansible safe to run in cron jobs and CI pipelines. The indication of a changed state *publish* is irrelevant.

Task 2: Create Two Groups (10 Minutes)

Same two groups you built in Lab 3 (Bad_IP_List, Good_IP_List), now declarative:

```
cat > 02-groups.yml <<'EOF'
- name: Lab 5 Task 2 - two groups
  connection: httpapi
  hosts: checkpoint_mgmt
  gather_facts: false
  collections: [checkpoint.mgmt]
  tasks:
    - { name: Bad_IP_List, cp_mgmt_group: { name: Bad_IP_List, state:
present } }
    - { name: Good_IP_List, cp_mgmt_group: { name: Good_IP_List, state:
present } }
    - { name: Publish, cp_mgmt_publish: {} }
EOF
ansible-playbook -i inventory.ini 02-groups.yml
```

```
ubuntu@AI-Ubuntu:~/lab5$ cat > 02-groups.yml <<'EOF'
- name: Lab 5 Task 2 - two groups
  connection: httpapi
  hosts: checkpoint_mgmt
  gather_facts: false
  collections: [checkpoint.mgmt]
  tasks:
    - { name: Bad_IP_List, cp_mgmt_group: { name: Bad_IP_List, state: present } }
    - { name: Good_IP_List, cp_mgmt_group: { name: Good_IP_List, state: present } }
    - { name: Publish, cp_mgmt_publish: {} }
EOF
ansible-playbook -i inventory.ini 02-groups.yml

PLAY [Lab 5 Task 2 - two groups] *****

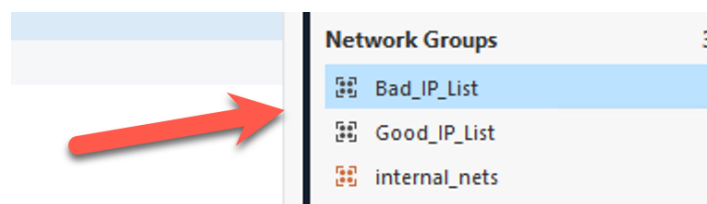
TASK [Bad_IP_List] *****
changed: [sms]

TASK [Good_IP_List] *****
changed: [sms]

TASK [Publish] *****
changed: [sms]

PLAY RECAP *****
sms : ok=3  changed=3  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
```

Both groups appear in SmartConsole → Network Objects → Groups. Empty groups are valid — a SOAR or threat-feed integration can populate them later.

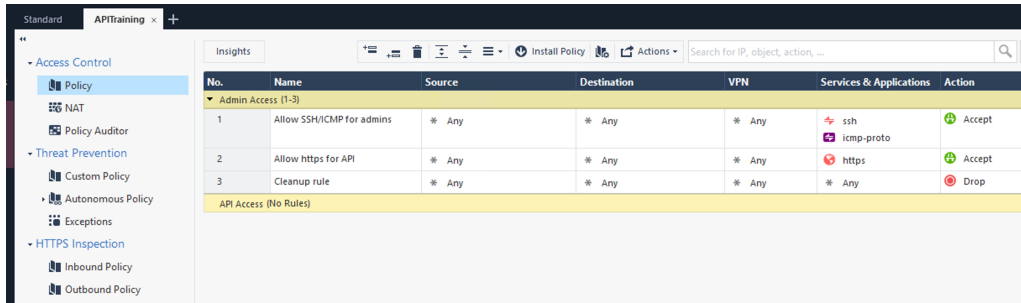


Task 3: Policy Package, Sections, and Rules (20 Minutes)

Same APITraining package from Lab 3 Task 3, now in YAML:

```
cat > 03-policy.yml <<'EOF'
- name: Lab 5 Task 3 - APITraining package
  connection: httpapi
  hosts: checkpoint_mgmt
  gather_facts: false
  collections: [check_point.mgmt]
  vars:
    layer: "APITraining Network"
  tasks:
    - name: Create package
      cp_mgmt_package: { name: APITraining, access: true, threat_prevention:
true, state: present }
    - name: Admin Access section
      cp_mgmt_access_section: { layer: "{{ layer }}", name: "Admin Access",
position: top, state: present }
    - name: API Access section
      cp_mgmt_access_section: { layer: "{{ layer }}", name: "API Access",
position: bottom, state: present }
    - name: SSH/ICMP rule
      cp_mgmt_access_rule:
        layer: "{{ layer }}"
        name: "Allow SSH/ICMP for admins"
        position: 1
        source: [Any]
        destination: [Any]
        service: [ssh, icmp-proto]
        action: Accept
        state: present
    - name: HTTPS rule
      cp_mgmt_access_rule:
        layer: "{{ layer }}"
        name: "Allow https for API"
        position: 2
        source: [Any]
        destination: [Any]
        service: [https]
        action: Accept
        state: present
    - { name: Publish, cp_mgmt_publish: {} }
EOF
ansible-playbook -i inventory.ini 03-policy.yml
```

Verify in SmartConsole → Security Policies → APITraining: two sections, one rule each. Compare to the Postman bodies you wrote in Lab 3 — that's Ansible's pedagogical win: one file, version-controllable, idempotent.



No.	Name	Source	Destination	VPN	Services & Applications	Action
Admin Access (1-3)						
1	Allow SSH/ICMP for admins	* Any	* Any	* Any	ssh, icmp-proto	Accept
2	Allow https for API	* Any	* Any	* Any	https	Accept
3	Cleanup rule	* Any	* Any	* Any	* Any	Drop
API Access (No Rules)						

Task 4: Install Policy (5 Minutes)

```
cat > 04-install.yml <<'EOF'
- name: Lab 5 Task 4 - install policy on GW
  connection: httpapi
  hosts: checkpoint_mgmt
  gather_facts: false
  collections: [checkpoint.mgmt]
  tasks:
    - cp_mgmt_install_policy: { policy_package: Standard, access: true,
  targets: [GW] }
EOF
ansible-playbook -i inventory.ini 04-install.yml
```

SmartConsole → Logs & Monitor → Open Tasks shows the install in progress.

Cleanup

```
cat > 99-cleanup.yml <<'EOF'
- name: Lab 5 cleanup
  connection: httpapi
  hosts: checkpoint_mgmt
  gather_facts: false
  collections: [checkpoint.mgmt]
  tasks:
    - { name: drop host, cp_mgmt_host: { name: lab-host-01, state: absent } }
    - { name: drop bad, cp_mgmt_group: { name: Bad_IP_List, state: absent } }
    - { name: drop good, cp_mgmt_group: { name: Good_IP_List, state: absent } }
    - { name: drop pkg, cp_mgmt_package: { name: APITraining, state: absent } }
    - { name: publish, cp_mgmt_publish: {} }
EOF
ansible-playbook -i inventory.ini 99-cleanup.yml
```

Checkpoint (5 Minutes)

For each scenario, write down which tool you would recommend — `mgmt_cli`, Postman, or Ansible — and one sentence saying why.

1. An on-call engineer needs to add three hosts and install policy at 3 AM. Speed matters; the change is one-time.
2. A customer is in a CISO meeting and wants to see what the API can do. The team has used Postman before.
3. A SOC team wants nightly enforcement of a threat feed: members must be reflected as a group; if already current, do nothing.
4. An SE builds a new policy package in their lab, version-controls it, and needs to reproduce it on a customer's pre-prod environment.

Expected directions: (1) `mgmt_cli` — one-shot speed; (2) Postman — visual demo; (3)

Ansible — idempotency is the whole game; (4) Ansible — version control + reproducibility.

References

1. **check_point.mgmt collection:** galaxy.ansible.com/check_point/mgmt
2. **Module reference:** docs.ansible.com/ansible/latest/collections/check_point/mgmt/
3. **Upstream:** github.com/CheckPointSW/CheckPointAnsibleMgmtCollection
4. **Real-world playbooks (Khalid's repo):** github.com/alshawwaf/Ansible_Projects — full demo policy, dynamic policy demo, Gaia provisioning, health checks.
5. **Deep-dive workshop:** [Workshops/Ansible-Workshop/](#) — 5 lab guides covering install, tasks, roles, AWX/Tower.
6. **Generic API escape hatch:** see the `cp_mgmt_generic_api` module and the standalone Generic API lab document.
7. **Gaia + Ansible:** the `check_point.gaia` collection drives the Gaia API (interfaces, routes, DNS, hostname, snapshots, etc.) using the same `httpapi` connection pattern you used in this lab — just a different module set (`cp_gaia_hostname`, `cp_gaia_dns`, `cp_gaia_physical_interface`, etc.) and the Gaia API user (Lab 4 Task 1) instead of the Mgmt admin. Working examples in [Scripts/ansible_gaia/](#) (~45 playbooks) and at galaxy.ansible.com/check_point/gaia.